

Lox Interpreter Architecture

A C++ Implementation

April 24, 2026

Contents

1	Introduction	3
2	Phase I: Lexical Tokenization	4
2.1	The Scanner Implementation	4
3	Phase II: Abstract Syntax Trees (AST)	6
3.1	Building the Tree in C++	6
4	Phase III: Recursive Descent Parsing	9
4.1	Parsing Precedence Rules	9
5	Phase IV: Tree-Walk Execution Engine	11
5.1	Dynamic Typing with std::variant	11
6	Phase V: Logic and Control Flow Branches	12
6.1	If Statements	12
7	Future Extensions	14
7.1	Automatic Differentiation via AST	14
7.2	Tensor Language Support	14
8	Conclusion	15

Chapter 1

Introduction

Building a programming language from scratch is a fantastic way to understand how computers actually read, parse, and execute code. For this project, I built a tree-walking interpreter in C++ that can run the Lox programming language.

The primary goal was to take raw text from a source file (like ‘test.lox’) and run it through a standard compilation pipeline natively. That means writing a Scanner to break strings down to tokens, a Parser to organize those tokens into a tree of expressions, and an Evaluator to actually run the logic.

This report documents exactly how I implemented each phase of the interpreter using standard C++17 features, memory management, and class inheritance.

Chapter 2

Phase I: Lexical Tokenization

The first step in any language implementation is dealing with raw text. The Interpreter doesn't know what the characters 'v', 'a', and 'r' mean when they are next to each other. The Scanner's job is to read these characters sequentially and chunk them into meaningful "Tokens" (like keywords, identifiers, or punctuation).

2.1 The Scanner Implementation

I created a 'Scanner' class that takes the raw source code string and iterates over it. Inside my 'scanToken()' method, I handle basic matching for spaces and single characters. However, things get tricky when dealing with operators like '==' or '<=', because the scanner has to look ahead to decide if it's one token or two.

This idea is known in compiler design as the "Maximal Munch" principle. The scanner tries to consume as many characters as possible to make the longest valid token. Here is exactly how I implemented that logic in my project:

```
// two-character operators
string twoChar = string(1, c) + peek();
auto it2 = twoCharTokens.find(twoChar);
if (it2 != twoCharTokens.end()) {
    advance();
    addToken(it2->second, twoChar);
    return;
}

// single-character tokens
```

```
auto it1 = singleCharTokens.find(c);
if (it1 != singleCharTokens.end()) {
    addToken(it1->second, string(1, c));
    return;
}
```

Instead of passing character by character blindly, I merge the current character ‘c’ with the next character using my ‘peek()’ helper to form a ‘twoChar’ string. I then check if this combination exists in my ‘twoCharTokens’ map (which contains operators like ‘==’ and ‘!=’). If it does, I immediately call ‘advance()’ to consume the second character and register the compound token.

If it doesn’t match a two-character operator, it falls through to check the ‘singleCharTokens’ map. This guarantees strings like ‘<=’ are always correctly prioritized over ‘<’ and ‘=’.

Chapter 3

Phase II: Abstract Syntax Trees (AST)

Once the scanner has generated a flat list of tokens, the tokens need to be organized. A flat list doesn't maintain any concept of the order of operations (like doing multiplication before addition). To solve this, I designed an Abstract Syntax Tree (AST).

3.1 Building the Tree in C++

Because different expressions require different data (a Literal just needs a value, but a Binary expression needs a left node, a right node, and an operator), I chose to represent the AST using C++ structs and inheritance.

I defined a base interface called 'Expr' inside 'Expression.hpp':

```
struct Expr {  
    // interface  
    virtual ~Expr() = default;  
    virtual string toString() = 0;  
    virtual LoxValue evaluate() = 0;  
};
```

Every type of expression inherits from this base 'Expr' struct. For example, my 'Grouping' expression is defined like this:

```
struct Grouping : Expr {  
public:  
    Expr* expr;  
    Grouping(Expr* expr) : expr(expr) {}  
};
```

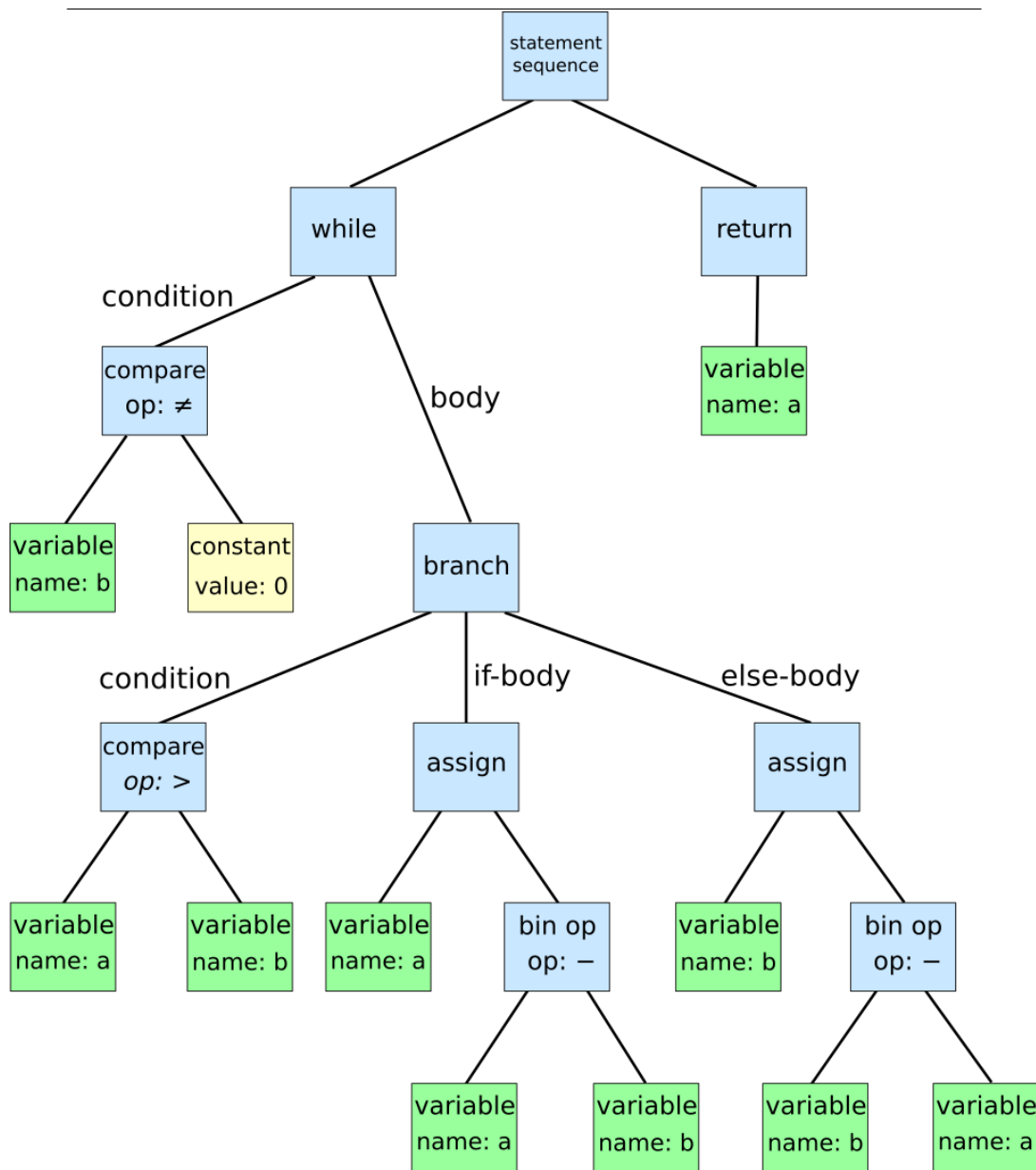


Figure 3.1: Visual representation of the AST branching structure mapping out logic.

```

string toString() override {
    return "(group " + expr->toString() + ")";
}

LoxValue evaluate() override {
    return expr->evaluate();
}
};

```

Notice how the ‘Grouping’ expression stores another ‘Expr*’ pointer inside it. This is what allows the AST to be recursive. A grouping node can contain a binary node, which can contain two literal nodes, creating the physical tree structure. I also mandated that every node must implement an ‘evaluate()’ function, which forms the core of how the code runs later on.

Chapter 4

Phase III: Recursive Descent Parsing

With the AST nodes defined, I needed a way to translate the tokens into those tree structures. I implemented a Recursive Descent Parser in ‘Parser.hpp’.

A Recursive Descent Parser is a top-down parser built out of a set of mutually recursive functions. Each rule in our grammar maps to a specific function in the C++ code. The lower the precedence of an operator, the higher it sits in the parser call stack.

4.1 Parsing Precedence Rules

Here is how I implemented the ‘expression()’ and ‘equality()’ rules:

```
Expr* expression() {
    return equality();
}

Expr* equality() {
    Expr* expr = comparison();

    while (match({TokenType::BANG_EQUAL, TokenType::EQUAL_EQUAL}))
    {
        Token op = previous();
        Expr* right = comparison();
        expr = new Binary(expr, op, right);
    }
    return expr;
}
```

When the parser tries to build an ‘expression’, it immediately hands it off to ‘equality()’. The ‘equality()’ function then attempts to parse the operands by calling down into ‘comparison()’.

If it finds an ‘==’ or ‘!=’ token (using the ‘match’ helper), it grabs the right-hand side, wraps both the left and right sides in a new ‘Binary’ AST node, and continues looping. This ensures that operators with lower precedence end up higher up in the true AST graph, which guarantees they physically evaluate last during the tree-walk phase. It’s a very elegant way to enforce PEMDAS without needing complicated lookup tables.

Chapter 5

Phase IV: Tree-Walk Execution Engine

After the Parser finishes doing its job, we are left with a single root ‘Expr*’ pointer that represents the entire file. Actually running the code is entirely handled by simply calling ‘evaluate()’ on that root pointer.

Because we used virtual functions across the C++ structs, C++ dynamic dispatch kicks in. Calling ‘evaluate()’ on a Binary node causes it to recursively call ‘evaluate()’ on its left and right children, reducing everything down until it hits the Literal leaves.

5.1 Dynamic Typing with std::variant

Lox is dynamically typed (meaning variables don’t declare if they are integers or strings), but C++ is strictly statically typed. To bridge this gap, I wrapped the return type of the evaluation cycle into a ‘std::variant’:

```
using LoxValue = variant<nullptr_t, double, string, bool,  
    shared_ptr<LoxCachable>, shared_ptr<LoxClass>, shared_ptr<  
    LoxInstance>>;
```

This allows my C++ functions to pass around a single data type that can safely hold numbers, strings, booleans, or objects. During evaluation, I just pull the underlying value out of the variant using ‘get<double>(val)’ if I need to do math, or throw an error if the types don’t match up.

Chapter 6

Phase V: Logic and Control Flow Branches

Programs wouldn't be very useful without conditional logic. Implementing 'if' and 'while' loops required creating custom 'Stmt' nodes in 'Stmt.hpp' that control exactly when to evaluate their child expressions.

6.1 If Statements

Unlike standard math, 'if' statements shouldn't evaluate both sides. Here is my exact implementation of conditional branching:

```
class ifStmt : public Stmt {
public:
    Expr* condition;
    Stmt* ifBranch = NULL;
    Stmt* elseBranch = NULL;

    void execute() override {
        LoxValue val = condition->evaluate();
        if (isTruthy(val)) {
            ifBranch->execute();
        } else if (elseBranch != NULL) {
            elseBranch->execute();
        }
    }
};
```

I evaluate the ‘condition’ first. Because of how I structured the C++ ‘execute()’ routine, the ‘elseBranch’ is completely ignored if the condition passes. This provides proper lazy evaluation natively inside C++.

Chapter 7

Future Extensions

The underlying framework defining this interpreter can be extended in highly practical ways. Moving forward, I plan to explore connecting these basic evaluation paradigms with advanced computational concepts used in AI and data processing.

7.1 Automatic Differentiation via AST

Neural networks rely on backpropagation, which is essentially deriving calculus functions. Since the interpreter already builds algebraic functions into a tree geometry, an extension could traverse the existing AST and mathematically mutate the nodes to produce a new AST representing the exact derivative of the function. This allows the framework to act as a symbolic math engine.

7.2 Tensor Language Support

The current ‘LoxValue’ variant supports standard floating point doubles. By extending this variant to hold N -dimensional arrays or standard library Vectors, the language can be retrofitted specifically for Linear Algebra. Extending operator overrides on Binary trees would then easily allow native matrix cross products cleanly mapped back to user syntax.

Chapter 8

Conclusion

Developing this project firmly reinforced the core tenets of software architecture. Working through Lexical Scanning, Recursive Top-Down Parsing, and dynamic C++ variants proved how layered abstractions turn basic string formatting into executable environments.